QReason: Query-Focused Decoupled Chain-of-Thought for Efficient Passage Reranking

Yang Zhang[1,2,3], Wenhan Liu[4], Qiannan Zhu[1,2,3]*, Mingming Li[5], Yuanfei Huang[1,2,3]
[1]School of Artificial Intelligence, Beijing Normal University
[2]Beijing Key Laboratory of Artificial Intelligence for Education
[3]Engineering Research Center of Intelligent Technology and Educational Application, Ministry of Education
[4]Gaoling School of Artificial Intelligence, Renmin University of China
[5]Institute of Information Engineering, Chinese Academy of Sciences
yangzsat@mail.bnu.edu.cn

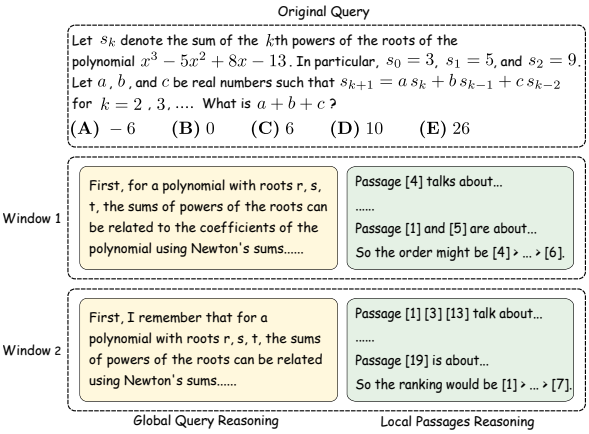
Abstract

Passage reranking plays a crucial role in information retrieval by refining the ordering of candidate passages to better reflect relevance. Existing listwise LLM rerankers with Chain-of-Thought (CoT) reasoning can handle complex queries effectively, but they suffer from substantial redundancy and high latency due to sliding-window strategies, which repeatedly generate highly similar CoTs. To address this, we propose QReason, a decoupled framework that separates query-focused reasoning from window-specific passage relevance assessment. Specifically, QReason introduces a dedicated rewriter that generates a ranking-oriented reasoning query once, capturing the query's core intent while avoiding redundant reasoning, and then reuses it across all windows with a non-reasoning reranker. The rewriter is trained via a two-stage process that first uses supervised fine-tuning with relevant-passage guidance through semantic evidence to produce deeply grounded, query-focused CoTs. It then applies reinforcement learning to align CoT generation with both the inference-time setting and the reranking objective, optimizing listwise metrics and passage-level discrimination to produce reusable reasoning chains for reranking. Experiments on the BRIGHT benchmark demonstrate that QReason significantly reduces redundant reasoning, achieves ranking performance comparable to or better than strong reasoning-based rerankers, and outperforms existing query rewriting models.

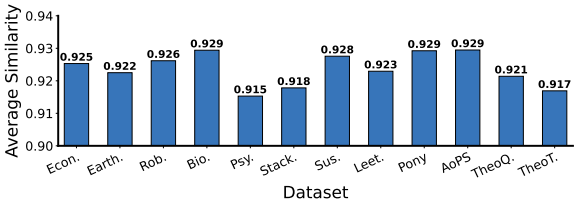
1 Introduction

Passage reranking is a critical stage in Information Retrieval, aiming to produce a finer-grained ordering of retrieved passages. Listwise reranking methods based on Large Language Models (LLMs) (Sun et al., 2023; Ma et al., 2023; Pradeep et al., 2023b,a; Liu et al., 2025a,b) have attracted substantial attention because they are able to model relative relationships among passages for accurate ranking. More recently, researchers have further introduced the reasoning capability of LLMs into reranking (Weller et al., 2025; Yang et al., 2025b; Zhuang et al., 2026; Liu et al., 2026c), explicitly generating CoT (Wei et al., 2022) during the ranking process to handle relevance judgment for complex queries. However, due to the limitation of context window size, these methods generally rely on sliding-window strategies (Sun et al., 2023), which require repeated CoT generation for the same query across multiple windows, thereby causing substantial computational redundancy and increased response latency.

We conduct an analysis of existing approaches to identify CoT redundancy, and observe that the



(a) An example of query CoT redundancy across windows.



(b) Similarity of query CoTs among windows across datasets.

Figure 1: Illustration of CoT redundancy in listwise rerankers.

*Corresponding author.

Proceedings of the 2026 Conference on Empirical Methods in Natural Language Processing, pages 30902–30925
October 24-29, 2026 ©2026 Association for Computational Linguistics

generated reasoning traces generally consist of two parts: global query reasoning and local passages reasoning, as illustrated in Figure 1(a). The query reasoning focuses on interpreting the core intent of the query and inferring the characteristics of potentially relevant passages in the global view. The passages reasoning analyzes the passages within the current window and determines whether they are relevant to the query. The former is independent of the specific passages in each window and already captures general reasoning about the characteristics of relevant passages, yet existing methods still regenerate the full reasoning trace for each window, resulting in substantial redundancy. To better quantify this phenomenon, we further analyze the reasoning traces produced by the representative reasoning-based reranking method ReasonRank (Liu et al., 2026c) on the BRIGHT benchmark (Su et al., 2025), and find that, for the same query, the average semantic similarity between reasoning traces generated in different windows exceeds 90%, as measured using BGE-m3 embeddings (Chen et al., 2024), as shown in Fig. 1(b). This high similarity indicates that most of the reasoning trace is repetitive and could, in principle, be reused across windows rather than recomputed.

Motivated by these insights, we propose QReason, a decoupled framework separating global query reasoning from window-specific reasoning by rewriting the original query into a reasoning query, i.e., a CoT-style reformulation of the query. The generated reasoning query is subsequently reused across windows by a non-reasoning reranker to eliminate redundant reasoning while preserving strong ranking performance. Therefore, our core objective is to train a rewriter that transforms the original query into a ranking-oriented reasoning query. To this end, we introduce a two-stage optimization framework. In the first stage, the rewriter is fine-tuned on the pairs of queries and their gold relevant-passages, enabling it to better capture query intent and generate grounded reasoning queries. In the second stage, the rewriter is further refined through reinforcement learning with two complementary rewards: a listwise reward that directly aligns rewriting with reranking quality, and a pointwise reward that constrains the generated reasoning query to remain supported by gold relevant-passages. Together, the two-stage optimization enables the rewriter to produce a query-focused and reusable reasoning query that efficiently guides reranking across candidate passages. Experiments on real-world datasets show that QReason consistently improves non-reasoning rerankers, and its strongest instantiation with Qwen3.5-35B-A3B (Qwen Team, 2026a) achieves superior ranking accuracy and computational efficiency compared with state-of-the-art reasoning-based rerankers.

In summary, our paper has three contributions:

- **Analysis of CoT Redundancy**. We reveal that reasoning-aware reranking generates highly similar query-level Chain-of-Thought across sliding windows, introducing substantial redundant reasoning that can be reused instead of repeatedly recomputed.
- **Novel QReason Framework**. We propose a decoupled architecture integrating a query-focused rewriter and a lightweight non-reasoning reranker, which reduces computational overhead while preserving strong ranking performance.
- **Two-Stage Rewriter Optimization**. We optimize the rewriter via supervised fine-tuning followed by reinforcement learning, yielding accurate, ranking-aligned reasoning queries for efficient and effective reranking.

2 Related Work

Reasoning-Intensive LLM Reranking In recent years, large language models have been increasingly adopted for reranking in information retrieval due to their strong semantic modeling ability (Zhuang et al., 2023; Pradeep et al., 2023b,a; Qin et al., 2024; Zhu et al., 2026; Sharifymoghaddam et al., 2025; Liu et al., 2026d). Nevertheless, prior studies (Su et al., 2025) have shown that existing LLM rerankers still perform poorly on reasoning-intensive IR tasks. Unlike conventional IR tasks that mainly rely on keyword and semantic matching, reasoning-intensive IR tasks require deeper query understanding and multi-step reasoning. To address this challenge, recent work has incorporated explicit reasoning into the reranking process. For instance, Rank-K (Yang et al., 2025b) and Rank1 (Weller et al., 2025) distill reasoning traces to supervise reranker fine-tuning, while ReasonRank (Liu et al., 2026c), Rank-r1 (Zhuang et al., 2026), and REARANK (Zhang et al., 2025a) optimize rerankers using GRPO (Shao et al., 2024). Although these approaches improve effectiveness on reasoning-intensive IR tasks, they typically require the model to explicitly generate reasoning

traces during inference, resulting in considerable computational cost and substantial redundancy.

Reasoning-Enhanced Query Rewriting Query rewriting has been widely adopted to improve first-stage retrieval by reformulating under-specified or ambiguous queries (Carpineto and Romano, 2012; Wang et al., 2023; Jagerman et al., 2023; Li et al., 2023). Recently, researchers have begun to leverage the reasoning capabilities of LLMs to address complex IR tasks. For example, some methods (Su et al., 2025) directly prompt large-scale LLMs, such as GPT-4 (OpenAI, 2023) or LLaMA3-70B-Instruct (Grattafiori et al., 2024), to generate reasoning-based rewrites, while others train smaller rewriters, such as DeepRetrieval (Jiang et al., 2025) and TongSearch (Qin et al., 2025), using reinforcement learning to endow them with reasoning capabilities. Although these approaches have demonstrated strong effectiveness for retrieval, they are primarily designed for the retrieval stage and do not transfer well to reranking.

3 Task Formulation

Given a query q and candidate passages $\mathcal{P}$, listwise reranking under context-length constraints divides $\mathcal{P}$ into overlapping windows $\mathcal{W} = \{W_1, \ldots, W_K\}$. Existing reasoning-based rerankers process each window independently:

$$c_j = g(q, W_j), \quad \pi_j = f(q, W_j, c_j), \qquad (1)$$

where a new reasoning trace c_j is independently generated for each window W_j, and π_j denotes the permutation produced by reranker f, ordering the passages in W_j by their estimated relevance to the query. This leads to substantial redundancy.

QReason instead decouples reusable query reasoning from window-level ranking:

$$q' = \mathcal{R}\mathcal{W}(q), \quad \pi_j = f(q, W_j, q'). \qquad (2)$$

Here, the rewriter generates a reranking-oriented reasoning query q' once, which is then reused across all windows by a non-reasoning reranker.

4 Method

As discussed above, an effective way to reduce redundancy is to externalize global query reasoning as a query rewrite that is independent of the local candidate passages, so that it can be reused across sliding windows without repeated generation. Though many retrieval-stage query rewriting methods notably boost retrieval performance, they cannot be directly adopted for reranking. Motivated by Li et al. (2024), we argue that rewriting for reranking differs from rewriting for retrieval in both objectives and constraints. Since rerankers jointly process the query and documents under a precision-oriented objective, they are more sensitive to fine-grained query-document interactions and to additional content introduced by rewriting. Additional content that may be harmless in retrieval can therefore deviate from the core ranking intent, causing false relevance signals and impairing ranking performance. Our experiments show that, although rewrites from GPT-4 and TongSearch are effective for retrieval, their effects become unstable when applied to reranking.

Building on this insight, we propose QReason, a decoupled framework that extracts global query reasoning by a *rewriter* as a CoT-style reasoning query and reuses it across windows in the non-reasoning reranker to eliminate redundant computation. The *rewriter* is first trained via supervised fine-tuning on (query, relevant-passages) pairs to capture intent accurately, and then refined with reinforcement learning using both listwise and pointwise rewards to align with inference-time setting and reranking objectives.

4.1 Relevance-Grounded Supervised Tuning

The rewriter in QReason is expected to generate a reasoning query that captures the underlying intent and integrates relevant passage evidence. To realize this goal, we argue that gold relevant-passages serve as the most reliable supervision signal, since they clearly define the critical query facets required for accurate relevance judgment. Accordingly, by introducing gold relevant-passages into the Relevance-Grounded Supervised Tuning (RGST) stage, we reformulate the rewriting task from an unconstrained query expansion into a grounded reasoning task guided by relevance information.

Formally, given a query q and its gold relevant-passages $\mathcal{P}^+$, we leverage a powerful teacher model (e.g., DeepSeek-R1) to synthesize the reasoning query q', which is specifically prompted to distill the reasoning chain connecting q to $\mathcal{P}^+$, highlighting the core intent required for precise passage ranking. Using the synthesized reasoning queries, we employ supervised fine-tuning (SFT) to train the rewriter, where each training sample is an input-output pair as $([q; \mathcal{P}^+], q')$. The opti-

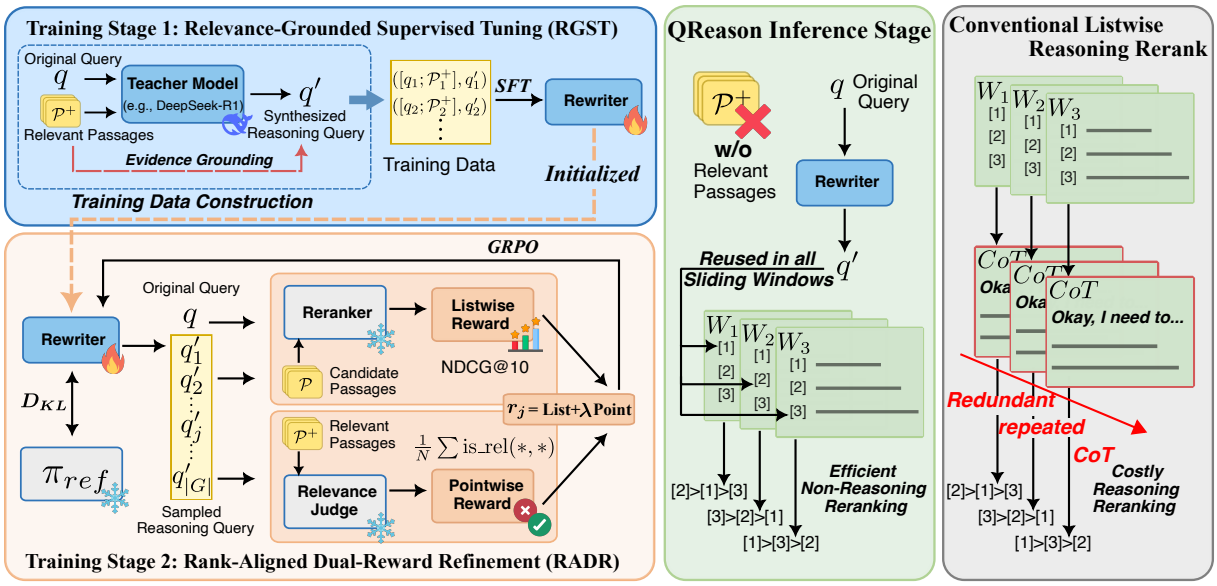


Figure 2: Overall framework of QReason, including two-stage training and inference-time comparison with conventional listwise reasoning reranker.

mization goal is the standard next-token prediction objective:

$$\mathcal{L}_{\text{RGST}} = -\sum_{t=1}^{|q'|} \log p_\theta\left(q'_t \mid q, \mathcal{P}^+, q'_{<t}\right), \quad (3)$$

where p_θ represents the rewriter parameterized by θ.

4.2 Rank-Aligned Dual-Reward Refinement

Although RGST guides the rewriter to generate evidence-grounded reasoning queries that capture key relevance patterns, there remains a training–inference discrepancy, as $\mathcal{P}^+$ is used during training but unavailable at inference time. Moreover, RGST does not directly optimize downstream reranking performance. QReason addresses these limitations through Rank-Aligned Dual-Reward Refinement (RADR), which introduces a reinforcement learning framework with task-specific rewards, including listwise rewards for overall ranking quality and pointwise rewards for passage-level discrimination. This stage directly aligns query generation with both the inference-time setting and the reranking objective, encouraging the rewriter to produce queries that are grounded and effective for precise ranking.

For each query q, we sample a group of $|G|$ reasoning queries from the current rewriter policy:

$$\mathcal{G}(q) = \{q'_1, q'_2, \ldots, q'_{|G|}\}, \qquad q'_j \sim \pi_\theta(\cdot \mid q).$$

Each sampled reasoning query q'_j is then evaluated by rewards tailored to reranking.

Listwise reward. The listwise reward aligns query generation with the downstream reranking objective. Specifically, for each sampled reasoning query q'_j in $\mathcal{G}(q)$, we feed the original query q, the rewritten reasoning query q'_j, and the candidate passages into a fixed *reranker* to obtain a reranked permutation π_j. The listwise reward is given by the NDCG@10 score of the resulting ranking, i.e.,

$$\text{List}(q'_j) = \text{NDCG@10}(\pi_j, \mathcal{Y}_q),$$

where $\mathcal{Y}_q$ denotes the relevance labels of the candidate passages for query q.

Pointwise reward. Optimizing only the listwise signal may cause the rewriter to overfit to the preference of a single fixed reranker, especially when that reranker has limited ranking capacity. To prevent the generated reasoning query from drifting away from the true semantic scope of relevant evidence, we introduce a pointwise reward based on gold relevant-passages, which constrains the rewrite to remain evidence-supported and avoids degenerate generations that improve reranker scores while being weakly related to actual relevance evidence.

Let $\mathcal{P}^+(q) = \{p^+_{q,i}\}_{i=1}^{N_q}$ denote the set of gold relevant passages for query q, where N_q denotes the number of judged relevant passages for q. For each sampled reasoning query q'_j, we use a *relevance judge* to independently determine whether q'_j is semantically consistent with each p^+_i. The

pointwise reward is defined as

$$\mathrm{Point}(q'_j) = \frac{1}{N_q} \sum_{i=1}^{N_q} \mathrm{is_rel}(q'_j, p^+_{q,i}),$$

where $\mathrm{is_rel}(q'_j, p^+_{q,i}) \in \{0, 1\}$ is a binary judgment function that returns 1 if the judge considers q'_j relevant to $p^+_{q,i}$, and 0 otherwise.

The final reward for each sampled reasoning query q'_j is defined as

$$r_j = \mathrm{List}(q'_j) + \lambda\, \mathrm{Point}(q'_j),$$

where λ controls the trade-off between direct ranking optimization and relevance-grounded semantic constraint.

Following GRPO (Shao et al., 2024), rewards are normalized within each sampled group to obtain the relative advantage $\hat{A}_j$, which is then used to optimize the rewriter with the clipped objective and KL regularization:

$$\begin{aligned}\mathcal{L}_{\mathrm{RADR}}(\theta) = -\frac{1}{|G|}\sum_{j=1}^{|G|}\frac{1}{|q'_j|}\sum_{t=1}^{|q'_j|}\Bigg[\min\Big(\rho_{j,t}(\theta)\hat{A}_j,\\ \mathrm{clip}\big(\rho_{j,t}(\theta),\,1-\varepsilon,\,1+\varepsilon\big)\hat{A}_j\Big)\\ -\beta D_{\mathrm{KL}}\Big(\pi_\theta(\cdot \mid s_{j,t}) \,\|\, \pi_{\mathrm{ref}}(\cdot \mid s_{j,t})\Big)\Bigg],\\ \rho_{j,t}(\theta) = \frac{\pi_\theta(q'_{j,t} \mid s_{j,t})}{\pi_{\theta_{\mathrm{old}}}(q'_{j,t} \mid s_{j,t})}.\end{aligned} \tag{4}$$

4.3 Training and Inference

Training Stage. In **RGST**, we initialize the rewriter from Qwen2.5-7B-Instruct and optimize it using the causal language modeling objective in Eq. (3). In **RADR**, we continue reinforcement learning from the RGST-trained rewriter. During rollout, the gold relevant-passages $\mathcal{P}^+$ are hidden from the policy model and used only for reward computation. We optimize the rewriter using the clipped GRPO objective in Eq. (4).

Inference Stage. At inference time, the rewriter operates autonomously and generates the reasoning query q' from the original query q alone. Although $\mathcal{P}^+$ is unavailable at this stage, it acts as a critical supervisory anchor that makes relevance-grounded reasoning observable during training, allowing the model to internalize the relevance patterns revealed by $(q, \mathcal{P}^+)$ pairs. In this sense, $\mathcal{P}^+$ serves as a supervisory scaffold that is removed after training. The reranker then takes both q and q', with q' directly appended to q, and performs non-reasoning ranking.

5 Experiment

5.1 Dataset

Training Data Construction. Our training data is constructed based on the public `reasonrank_data_13k` dataset released by ReasonRank (Liu et al., 2026c). We use DeepSeek-R1 (Guo et al., 2025) as the teacher model to construct the RGST training data.

Evaluation Benchmark. We evaluate our method on BRIGHT (Su et al., 2025), a benchmark for reasoning-intensive retrieval. Since relevant documents often require deeper reasoning beyond lexical or shallow semantic matching, BRIGHT is well suited for evaluating complex reranking. We use NDCG@10 as the evaluation metric. In addition to BRIGHT, we also report supplementary results on FreshStack (Thakur et al., 2025) and NeuCLIRBench (Lawrie et al., 2025) in Appendix D.3 to further examine the effectiveness of our method.

5.2 Baselines

The baselines used in our experiments can be divided into two groups: task-specific rerankers and general LLM rerankers.

Task-Specific Rerankers. We include several models specifically trained or optimized for reranking, including RankT5 (3B) (Zhuang et al., 2023), RankZephyr (7B) (Pradeep et al., 2023b), Rearank (7B) (Zhang et al., 2025a), Rank-R1 (7B and 14B) (Zhuang et al., 2026), Rank1 (7B and 32B) (Weller et al., 2025), Rank-K (32B) (Yang et al., 2025b) and ReasonRank (7B and 32B) (Liu et al., 2026c). Among them, Rearank (7B), Rank-R1 (7B and 14B), Rank1 (7B and 32B), Rank-K (32B), and ReasonRank (7B and 32B) are reasoning rerankers, while RankT5 (3B) and RankZephyr (7B) are non-reasoning rerankers.

General LLM Rerankers. We use Qwen2.5-32B-Instruct (Yang et al., 2024), Qwen3-30B-A3B-Instruct-2507 (Yang et al., 2025a), and Qwen3.5-35B-A3B (Qwen Team, 2026a) as rerankers, all of which perform non-reasoning reranking. On top of these rerankers, we compare multiple rewriters or rewriting methods, including GPT-4 (Ope-

Table 1: Performance comparison on BRIGHT. The best score is shown in **bold** and the second best is underlined. Rows denoted as "with X" use queries from X for reranking, where X can be the original queries or rewritten queries generated by GPT-4, DeepRetrieval, TongSearch, or QReason. All reranker baseline results are reproduced by us under the same experimental environment.

	Econ.	Earth.	Rob.	Bio.	Psy.	Stack.	Sus.	Leet.	Pony	AoPS	TheoQ.	TheoT.	Avg
ReasonIR	32.43	44.19	20.82	43.49	39.70	30.96	27.34	**32.33**	19.23	8.14	32.84	36.41	30.66
Task-Specific Rerankers													
RankT5 (3B)	11.28	22.17	11.25	11.59	11.45	10.08	17.92	27.41	38.98	9.33	18.27	11.32	16.75
RankZephyr (7B)	19.85	19.02	12.2	33.77	24.78	13.01	23.69	29.14	32.76	6.61	24.9	30.17	22.49
Rank1 (7B)	25.88	38.74	16.32	39.87	35.15	24.51	33.59	12.90	28.11	3.64	30.60	38.18	27.29
Rank-R1 (7B)	21.12	27.36	18.13	36.52	30.27	10.74	29.30	16.88	10.28	3.09	14.35	30.47	20.71
Rearank (7B)	30.46	41.35	28.55	47.08	40.72	26.10	36.49	31.25	22.24	7.76	30.17	40.52	31.89
ReasonRank (7B)	34.19	46.16	30.68	**56.71**	48.90	28.65	42.30	22.90	25.74	7.42	38.56	42.36	35.38
Rank-R1 (14B)	27.42	36.75	24.81	43.02	37.18	28.22	36.73	21.24	20.58	7.11	32.09	35.58	29.23
Rank1 (32B)	25.63	32.28	17.63	42.72	35.25	23.55	31.08	12.24	**41.10**	5.67	29.64	39.17	28.00
Rank-K (32B)	28.49	38.43	24.62	48.22	43.48	30.97	32.29	25.98	23.00	9.25	35.93	42.74	31.95
ReasonRank (32B)	34.49	47.16	30.97	54.65	**52.11**	33.99	**45.06**	27.77	19.99	7.94	38.49	44.83	36.45
General LLM Rerankers													
Qwen2.5-32B-Instruct													
with Original Queries	32.27	42.20	25.32	52.18	46.71	28.82	41.99	17.82	27.89	7.16	27.59	43.02	32.75
with GPT-4	30.59	45.01	31.08	52.72	41.38	28.21	37.09	22.34	23.50	6.80	35.35	41.66	32.98 (↑0.23)
with DeepRetrieval	30.51	43.59	25.63	51.01	47.99	29.39	41.65	19.49	28.02	7.18	27.16	42.11	32.81 (↑0.06)
with TongSearch (7B)	30.08	41.35	26.49	49.27	47.28	27.17	41.81	20.11	24.44	7.48	28.67	41.50	32.14 (↓0.61)
with **QReason**	35.70	44.40	29.44	51.46	48.39	31.11	42.56	18.06	34.06	9.66	32.51	44.86	35.18 (↑2.43)
Qwen3-30B-A3B-Instruct-2507													
with Original Queries	33.23	49.21	27.54	50.39	43.90	**34.22**	40.81	27.50	21.95	9.11	33.96	41.90	34.48
with GPT-4	29.33	46.33	28.09	47.72	42.94	28.03	35.80	25.86	18.29	9.47	30.80	39.82	31.87 (↓2.61)
with DeepRetrieval	32.17	48.33	30.24	51.15	46.30	32.86	42.69	29.18	20.13	**10.02**	31.78	41.50	34.70 (↑0.22)
with TongSearch (7B)	32.88	46.26	25.96	49.38	43.21	32.73	38.87	27.48	20.22	7.73	31.63	39.90	33.02 (↓1.46)
with **QReason**	**36.58**	**49.61**	28.39	52.83	45.80	33.33	43.98	28.35	19.68	9.09	35.04	43.73	35.53 (↑1.05)
Qwen3.5-35B-A3B													
with Original Queries	32.07	43.89	**32.06**	53.89	47.94	30.26	39.61	28.75	27.24	8.26	40.28	41.58	35.49
with GPT-4	30.35	47.83	29.95	55.74	45.27	31.18	40.36	30.26	27.56	8.43	39.65	43.15	35.81 (↑0.32)
with DeepRetrieval	32.34	45.35	30.49	53.47	46.82	29.66	40.53	27.30	24.57	8.73	41.37	43.15	35.32 (↓0.17)
with TongSearch (7B)	33.84	44.10	30.22	53.27	45.68	29.22	39.84	30.72	28.79	8.47	40.95	43.23	35.69 (↑0.20)
with **QReason**	35.34	46.63	30.46	56.44	47.12	29.52	44.05	31.55	25.70	9.17	**41.73**	**45.49**	**36.93** (↑1.44)

nAI, 2023), DeepRetrieval (Jiang et al., 2025), TongSearch (7B) (Qin et al., 2025), and our method. We do not pair rewritten queries with rerankers specifically trained for reranking, because these models are trained on original queries and are therefore adapted to the distribution of original-query inputs.

Following ReasonRank, we use the GPT-4-rewritten queries provided by BRIGHT as retrieval queries, and adopt ReasonIR (8B) (Shao et al., 2025) as the initial retriever. We then rerank the top-100 passages retrieved by ReasonIR.

5.3 Main Results

As shown in Table 1, QReason achieves the best overall performance on BRIGHT when paired with Qwen3.5-35B-A3B, reaching an average score of 36.93 and outperforming reasoning reranker baselines of a similar scale, including Rank1 (32B), Rank-K (32B), and the strongest baseline, ReasonRank (32B). This result is particularly notable because these baselines require generating reasoning traces multiple times during reranking, whereas QReason only needs to generate the reasoning query once and then performs non-reasoning reranking. The gain over ReasonRank (32B) therefore suggests that decoupling reusable query-level reasoning from window-specific ranking can reduce repeated reasoning generation and improve ranking efficiency, while still achieving performance comparable to, or even better than, the strongest reasoning reranking model.

QReason also delivers the most consistent improvements across different rerankers and is substantially more stable than existing rewriting baselines. Specifically, when applied to Qwen2.5-32B-Instruct, Qwen3-30B-A3B-Instruct-2507, and Qwen3.5-35B-A3B, QReason consistently improves the original-query setting by 2.43, 1.05, and 1.44 points, respectively. In contrast, the other rewriting methods show much less stable behavior. For example, GPT-4 drops by 2.61 points on Qwen3-30B-A3B-Instruct-2507, and TongSearch (7B) reduces performance on both Qwen2.5-32B-Instruct and Qwen3-30B-A3B-Instruct-2507.

Table 2: Ablation study of QReason on BRIGHT using Qwen2.5-32B-Instruct as the reranker.

Variant	Setting	Avg.
QReason	Full	**35.18**
RGST and RADR Training	Only RGST	33.50
	Only RADR	33.91
Relevant-Passages Guidance	RGST w/o $\mathcal{P}^+$	32.98
Reward Design	w/o Pointwise Reward	33.85
	w/o Listwise Reward	33.90

These results indicate that generic rewriting does not reliably benefit reranking, whereas QReason produces ranking-oriented reasoning queries that transfer more robustly across different rerankers.

5.4 Ablation Study

Table 2 reports the ablation results of QReason on BRIGHT using Qwen2.5-32B-Instruct as the reranker.

The Effectiveness of RGST and RADR Training. A clear performance drop is observed when either training stage is removed. Using only RGST leads to a decline of 1.68 points in Avg. NDCG@10, while using only RADR results in a drop of 1.27 points. This shows that the two stages play complementary roles. RGST provides stable evidence-grounded initialization for reasoning query generation, while RADR further aligns the rewritten queries with downstream reranking objectives.

The Effectiveness of Relevant-Passages Guidance in RGST. Excluding gold relevant-passages from RGST causes a significant degradation of 2.20 points. This result verifies that introducing $\mathcal{P}^+$ during supervised tuning is critical for grounding the rewriting process in explicit relevance evidence. Without such grounding, the generated reasoning queries are more likely to drift away from the key facets required for accurate ranking.

The Effectiveness of Reward Design in RADR. Removing either reward term in RADR also degrades performance. Specifically, removing the pointwise reward causes a drop of 1.33 points, while removing the listwise reward results in a decline of 1.28 points. These results indicate that the two rewards are complementary: the listwise reward directly optimizes ranking quality at the whole-list level, whereas the pointwise reward constrains the rewritten query to remain consistent with the gold relevant-passages.

5.5 Efficiency Analysis

The total generation cost of conventional reasoning listwise rerankers can be expressed as

$$G_{\text{reason}} = \sum_{j=1}^{K} \big(\ell(c_j) + \ell(\pi_j)\big) \approx K(R+O), \quad (5)$$

where $\ell(\cdot)$ denotes the output length, K is the number of sliding windows, and R and O denote the average lengths of the reasoning trace and ranking output, respectively. Likewise, the total generation cost of QReason is

$$G_{\text{QReason}} = \ell(q') + \sum_{j=1}^{K} \ell(\pi'_j) \approx R' + KO, \quad (6)$$

where $R' = \ell(q')$ is the length of the rewritten reasoning query. Compared with the conventional paradigm, QReason replaces repeated window-specific reasoning generation with a one-time rewriting step. In practice, with $R \approx R'$, the efficiency gain mainly comes from avoiding reasoning generation in the remaining $K-1$ windows.

It is worth noting that QReason appends q' to each ranking prompt, which slightly increases the input length of each window. However, this extra cost is introduced in the prefilling stage, whereas conventional reasoning rerankers repeatedly generate full reasoning traces through autoregressive decoding, which is inherently sequential and more latency-sensitive in practice.

We further compare the empirical efficiency of two QReason instantiations against ReasonRank (32B) on six BRIGHT subsets. As shown in Table 3, QReason yields substantial end-to-end efficiency gains over explicit reasoning reranking. After including the rewriting cost, QReason still achieves about $7.9\times$ speedup when using the Mixture-of-Experts (MoE) reranker Qwen3.5-35B-A3B and about $2.6\times$ speedup when using Qwen2.5-32B-Instruct, compared with ReasonRank (32B). These results show that QReason substantially alleviates the latency bottleneck of reasoning listwise rerankers.

5.6 Parameter Sensitivity

We study the weighting coefficient λ in RADR, which balances the listwise and pointwise rewards.

Table 3: End-to-end latency comparison on six BRIGHT subsets. For QReason, the total latency is measured end-to-end, including both the one-time query rewriting cost and the downstream reranking cost. Unit: seconds.

Dataset	ReasonRank (32B)	**QReason** + Qwen3.5-35B-A3B		**QReason** + Qwen2.5-32B-Instruct	
	Cost	Cost	Speedup	Cost	Speedup
economic	1056.08	163.83	6.45×	503.97	2.10×
biology	754.67	95.89	7.87×	277.92	2.72×
sustainable_living	820.03	130.01	6.31×	368.01	2.23×
leetcode	3385.11	315.04	10.75×	1193.75	2.84×
aops	1742.63	228.83	7.62×	663.01	2.63×
theoremqa_theorems	1188.31	205.18	5.79×	456.79	2.60×
avg	1491.14	**189.80**	**7.86**×	**577.24**	**2.58**×

Table 4: Case study of QReason and TongSearch rewrites. Green text denotes content aligned with the gold evidence, while red text denotes unsupported details introduced in the rewrite.

Query Intent	Gold Evidence	QReason Rewrite	TongSearch (7B) Rewrite
partition household food expenditures into BEA producing industries and attribute expenditures through an input-output framework.	input-output accounting framework; supply-use / make-use table system; industry-commodity mapping; expenditure attribution through production chains.	**Matched:** input-output accounting framework; supply-use / make-use table system; industry-commodity mapping. **Noise:** national accounting systems; expenditure survey methodology.	**Matched:** input-output accounting framework. **Noise:** Food-at-Home / Food-Away-from-Home mapping; direct allocation to 311FT, 445, and 722; assumed expenditure shares; splitting strategy.

We vary λ from 0.1 to 1.5, train the rewriter under the same setting, and evaluate the resulting rewritten queries using Qwen2.5-32B-Instruct as the reranker. The performance exhibits a clear non-monotonic trend: it first improves as λ increases and then declines when λ becomes too large, with the best result obtained at $\lambda = 0.6$. This indicates that a moderate pointwise reward can regularize the semantic scope of the rewritten query without weakening its downstream reranking utility.

5.7 Case Study

Table 4 presents a representative case for qualitatively comparing different rewritten queries. **Query Intent** summarizes the main information need in the original query, and **Gold Evidence** lists the core evidence reflected in the relevant passages. Under each rewritten query, **Matched** refers to content aligned with the gold evidence, whereas **Noise** refers to additional content not directly supported by the gold relevant-passages. Owing to space constraints, we report only the most salient phrases. Full case details are provided in Appendix E.

In the case shown in the table, the original query asks how household food expenditures can be attributed to BEA producing industries under an input-output framework for pollution-impact analysis. Accordingly, the gold evidence centers on the accounting structure needed for such attribution, including the input-output framework, the supply-use / make-use table system, and industry-commodity mapping. QReason includes the input-output framework, supply-use tables, and industry-level linkage structure emphasized in the gold evidence, together with some broad additional content such as national accounting systems and expenditure survey methodology. TongSearch also covers part of the same evidence space, but further introduces more specific allocation details, which are not explicitly supported by the gold evidence.

6 Conclusion

In this paper, we propose QReason, a decoupled reranking framework that reuses query-level reasoning across sliding windows. Existing reasoning-based rerankers repeatedly generate highly similar and redundant reasoning traces across sliding windows. QReason addresses this issue by training a rewriter to generate a ranking-oriented reasoning query through Relevance-Grounded Supervised Tuning and Rank-Aligned Dual-Reward Refinement. The generated reasoning query is then reused by non-reasoning rerankers across different windows. Experiments on BRIGHT demonstrate

that QReason consistently improves different LLM rerankers and achieves strong performance without redundant reasoning generation.

Limitations

QReason optimizes reasoning queries with listwise and pointwise relevance rewards, which do not explicitly capture multidimensional requirements such as coverage, faithfulness, and redundancy. Rubrics offer an interpretable way to specify such criteria (Liu et al., 2026a), and have recently been applied to document reranking (Liu et al., 2026b). Future work could incorporate query-specific rubric rewards into RADR to improve the controllability and robustness of reusable reasoning queries.

Ethical Considerations

This work focuses on query rewriting and passage reranking for information retrieval. It does not involve the collection of new user data, personal information, or sensitive attributes, and does not include human-subject studies or human annotation. All experiments are conducted on publicly available datasets and pretrained models, and we cite the corresponding sources. The main potential risk is that errors in rewriting or reranking may affect the order of retrieved passages and expose users to less accurate or less reliable information. This risk is common to retrieval and reranking systems, and the proposed method is intended for research use rather than direct deployment in high-stakes scenarios.

Acknowledgments

This work was supported by the National Natural Science Foundation of China No. 62472038 and No. 62437001.

References

Claudio Carpineto and Giovanni Romano. 2012. A survey of automatic query expansion in information retrieval. *ACM Comput. Surv.*, 44(1):1:1–1:50.

Jianlyu Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu. 2024. M3-embedding: Multi-linguality, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. In *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, Findings of ACL, pages 2318–2335. Association for Computational Linguistics.

Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. In *Advances in Neural Information Processing Systems*, volume 35, pages 16344–16359. Curran Associates, Inc.

Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, Amy Yang, Angela Fan, Anirudh Goyal, Anthony Hartshorn, Aobo Yang, Archi Mitra, Archie Sravankumar, Artem Korenev, Arthur Hinsvark, and 542 others. 2024. The llama 3 herd of models. *Preprint*, arXiv:2407.21783.

Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, and 175 others. 2025. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nat.*, 645(8081):633–638.

Rolf Jagerman, Honglei Zhuang, Zhen Qin, Xuanhui Wang, and Michael Bendersky. 2023. Query expansion by prompting large language models. *CoRR*, abs/2305.03653.

Pengcheng Jiang, Jiacheng Lin, Lang Cao, Runchu Tian, SeongKu Kang, Zifeng Wang, Jimeng Sun, and Jiawei Han. 2025. Deepretrieval: Hacking real search engines and retrievers with large language models via reinforcement learning. In *Second Conference on Language Modeling*.

Dawn J. Lawrie, James Mayfield, Eugene Yang, Andrew Yates, Sean MacAvaney, Ronak Pradeep, Scott Miller, Paul McNamee, and Luca Soldani. 2025. Neuclir-bench: A modern evaluation collection for monolingual, cross-language, and multilingual information retrieval. *CoRR*, abs/2511.14758.

Hang Li, Ahmed Mourad, Shengyao Zhuang, Bevan Koopman, and Guido Zuccon. 2023. Pseudo relevance feedback with deep language models and dense retrievers: Successes and pitfalls. *ACM Trans. Inf. Syst.*, 41(3):62:1–62:40.

Minghan Li, Honglei Zhuang, Kai Hui, Zhen Qin, Jimmy Lin, Rolf Jagerman, Xuanhui Wang, and Michael Bendersky. 2024. Can query expansion improve generalization of strong cross-encoder rankers? In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2024, Washington DC, USA, July 14-18, 2024*, pages 2321–2326. ACM.

Wenhan Liu, Jiajie Jin, Zhaoheng Huang, Tongyu Wen, Guanting Dong, Ziliang Zhao, Yutao Zhu, Zhicheng Dou, and Ji-Rong Wen. 2026a. The rules of the game: A survey of rubrics for large language models.

Wenhan Liu, Yu Lu, Qiaolin Xia, Hui Xu, Tong Zhao, Jian Xi, Yutao Zhu, Haijin Liang, Haibo Shi, Hao

Wang, and Zhicheng Dou. 2026b. Training documents reranker with search rubrics for deep research agent. *Preprint*, arXiv:2608.03527.

Wenhan Liu, Xinyu Ma, Weiwei Sun, Yutao Zhu, Yuchen Li, Dawei Yin, and Zhicheng Dou. 2026c. Reasonrank: Empowering passage ranking with strong reasoning ability. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2026, San Diego, California, United States, July 2-7, 2026*, pages 22007–22031. Association for Computational Linguistics.

Wenhan Liu, Xinyu Ma, Yutao Zhu, Lixin Su, Shuaiqiang Wang, Dawei Yin, and Zhicheng Dou. 2025a. Coranking: Collaborative ranking with small and large ranking agents. In *Findings of the Association for Computational Linguistics: EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 5098–5110. Association for Computational Linguistics.

Wenhan Liu, Xinyu Ma, Yutao Zhu, Ziliang Zhao, Shuaiqiang Wang, Dawei Yin, and Zhicheng Dou. 2025b. Sliding windows are not the end: Exploring full ranking with long-context large language models. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), ACL 2025, Vienna, Austria, July 27 - August 1, 2025*, pages 162–176. Association for Computational Linguistics.

Wenhan Liu, Yutao Zhu, Zhicheng Dou, and Yujia Zhou. 2026d. Demorank: Selecting effective demonstrations for large language models in ranking task. *ACM Trans. Inf. Syst.*, 44(3):61:1–61:25.

Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-shot listwise document reranking with a large language model. *CoRR*, abs/2305.02156.

OpenAI. 2023. GPT-4 technical report. *CoRR*, abs/2303.08774.

Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. 2023a. Rankvicuna: Zero-shot listwise document reranking with open-source large language models. *CoRR*, abs/2309.15088.

Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. 2023b. Rankzephyr: Effective and robust zero-shot listwise reranking is a breeze! *CoRR*, abs/2312.02724.

Xubo Qin, Jun Bai, Jiaqi Li, Zixia Jia, and Zilong Zheng. 2025. Reinforced query reasoners for reasoning-intensive retrieval tasks. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 21250–21263. Association for Computational Linguistics.

Zhen Qin, Rolf Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Le Yan, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2024. Large language models are effective text rankers with pairwise ranking prompting. In *Findings of the Association for Computational Linguistics: NAACL 2024, Mexico City, Mexico, June 16-21, 2024*, Findings of ACL, pages 1504–1518. Association for Computational Linguistics.

Qwen Team. 2026a. Qwen3.5: Towards native multimodal agents.

Qwen Team. 2026b. Qwen3.6-35B-A3B: Agentic coding power, now open to all.

Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: memory optimizations toward training trillion parameter models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC 2020, Virtual Event / Atlanta, Georgia, USA, November 9-19, 2020*, page 20. IEEE/ACM.

Rulin Shao, Rui Qiao, Varsha Kishore, Niklas Muennighoff, Xi Victoria Lin, Daniela Rus, Bryan Kian Hsiang Low, Sewon Min, Wen tau Yih, Pang Wei Koh, and Luke Zettlemoyer. 2025. ReasonIR: Training retrievers for reasoning tasks. In *Second Conference on Language Modeling*.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *CoRR*, abs/2402.03300.

Sahel Sharifymoghaddam, Ronak Pradeep, Andre Slavescu, Ryan Nguyen, Andrew Xu, Zijian Chen, Yilin Zhang, Yidi Chen, Jasper Xian, and Jimmy Lin. 2025. Rankllm: A python package for reranking with llms. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2025, Padua, Italy, July 13-18, 2025*, pages 3681–3690. ACM.

Hongjin Su, Howard Yen, Mengzhou Xia, Weijia Shi, Niklas Muennighoff, Han-yu Wang, Haisu Liu, Quan Shi, Zachary S. Siegel, Michael Tang, Ruoxi Sun, Jinsung Yoon, Sercan Ö. Arik, Danqi Chen, and Tao Yu. 2025. BRIGHT: A realistic and challenging benchmark for reasoning-intensive retrieval. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net.

Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is chatgpt good at search? investigating large language models as re-ranking agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 14918–14937. Association for Computational Linguistics.

Nandan Thakur, Jimmy Lin, Samuel Havens, Michael Carbin, Omar Khattab, and Andrew Drozdov. 2025.

Freshstack: Building realistic benchmarks for evaluating retrieval on technical documents. In *Advances in Neural Information Processing Systems*, volume 38, Main Conference. Curran Associates, Inc.

Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query expansion with large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pages 9414–9423. Association for Computational Linguistics.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, brian ichter, Fei Xia, Ed Chi, Quoc V Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837. Curran Associates, Inc.

Orion Weller, Kathryn Ricci, Eugene Yang, Andrew Yates, Dawn Lawrie, and Benjamin Van Durme. 2025. Rank1: Test-time compute for reranking in information retrieval. In *Second Conference on Language Modeling*.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025a. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, and 22 others. 2024. Qwen2.5 technical report. *CoRR*, abs/2412.15115.

Eugene Yang, Andrew Yates, Kathryn Ricci, Orion Weller, Vivek Chari, Benjamin Van Durme, and Dawn J. Lawrie. 2025b. Rank-k: Test-time reasoning for listwise reranking. *CoRR*, abs/2505.14432.

Le Zhang, Bo Wang, Xipeng Qiu, Siva Reddy, and Aishwarya Agrawal. 2025a. REARANK: reasoning re-ranking agent via reinforcement learning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, EMNLP 2025, Suzhou, China, November 4-9, 2025*, pages 2458–2471. Association for Computational Linguistics.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025b. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *CoRR*, abs/2506.05176.

Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2026. Large language models for information retrieval: A survey. *ACM Trans. Inf. Syst.*, 44(1):12:1–12:54.

Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. 2023. Rankt5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval, SIGIR 2023, Taipei, Taiwan, July 23-27, 2023*, pages 2308–2313. ACM.

Shengyao Zhuang, Xueguang Ma, Zheng Yao, Shuai Wang, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2026. Rank-r1: Enhancing reasoning in llm-based document rerankers via reinforcement learning. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '26, page 4419–4425, New York, NY, USA. Association for Computing Machinery.

A Evaluation Benchmark and Baselines

A.1 Evaluation Benchmark

In our experiments, we use BRIGHT (Su et al., 2025) as the evaluation benchmark. BRIGHT is a reasoning-intensive retrieval benchmark designed to evaluate whether retrieval systems can identify relevant documents beyond lexical or shallow semantic matching. It contains 1,384 real-world queries from diverse domains, including economics, psychology, mathematics, and coding, with queries derived from naturally occurring data and carefully curated human inputs. Unlike conventional retrieval benchmarks where surface-form or semantic similarity is often sufficient, BRIGHT requires models to infer implicit connections between the query and candidate documents. For example, resolving a coding query may require understanding the underlying program logic, while mathematical and scientific queries may require matching the query to relevant concepts, theorems, or explanatory evidence. Therefore, BRIGHT provides a challenging testbed for evaluating retrieval and reranking methods under reasoning-intensive scenarios.

A.2 Baselines

ReasonIR. ReasonIR-8B (Shao et al., 2025) is a LLaMA3.1-8B-based bi-encoder retriever specifically trained for reasoning-intensive retrieval, using synthetic reasoning-oriented data with challenging queries and hard negatives.

RankT5. RankT5 (Zhuang et al., 2023) is a T5-based pointwise reranker optimized with a ranking loss function, which scores each query-document pair independently for relevance estimation.

RankZephyr. RankZephyr (Pradeep et al., 2023b) is an open-source listwise LLM reranker distilled from GPT-3.5- and GPT-4-generated ranking data, and performs reranking by directly generating an ordered list of candidate passages.

Rearank. REARANK (Zhang et al., 2025a) is a Qwen2.5-7B-based listwise reasoning reranker trained with reinforcement learning, which explicitly generates reasoning before producing the final ranked list of candidate passages.

Rank1. Rank1 (Weller et al., 2025) is a pointwise reasoning reranker distilled from DeepSeek-R1 reasoning traces, which estimates relevance by generating a reasoning chain before making a true/false relevance judgment.

Rank-R1. Rank-R1 (Zhuang et al., 2026) is a setwise reasoning reranker trained with GRPO, which selects the most relevant passage from a candidate set and applies heap sort to construct the final ranking.

Rank-K. Rank-K (Yang et al., 2025b) is a QwQ-32B-based listwise reasoning reranker distilled from DeepSeek-R1, which generates reasoning traces and applies a sliding-window strategy during inference.

ReasonRank. ReasonRank (Liu et al., 2026c) is trained through a two-stage framework on high-quality reasoning-intensive data synthesized by DeepSeek-R1, consisting of a cold-start supervised fine-tuning (SFT) stage and a reinforcement learning (RL) stage. In the RL stage, it is optimized with a multi-view ranking reward. ReasonRank performs listwise reranking with a sliding-window strategy and achieves strong performance across multiple benchmarks.

DeepRetrieval. DeepRetrieval (Jiang et al., 2025) is an RL-based query generation method that trains LLMs through trial and error without supervised reference queries, using retrieval metrics as rewards to generate queries that improve retrieval performance.

TongSearch. TongSearch QR (Qin et al., 2025) is a family of small-scale language models for query reasoning and rewriting in reasoning-intensive retrieval, trained with reinforcement learning and a semi-rule-based reward function.

B Implementation Details

B.1 Training Data Construction

Our training data is constructed based on the public `reasonrank_data_13k`[1] dataset released by ReasonRank (Liu et al., 2026c). The `reasonrank_data_13k` dataset contains 13,549 training instances spanning ten source datasets: `math-theorem`, `sustainable_living`, `msmarco`, `biology`, `robotics`, `math-qa`, `leetcode`, `earth_science`, `stackoverflow`, and `economics`. We randomly sample 6,000 instances from the full dataset to construct the RGST data, and use the remaining 7,549 instances to build the RADR data.

B.2 Training Details

We provide the implementation details of the two training stages in QReason, including the training framework, hardware setup, key hyperparameters, data construction, and prompt templates.

RGST. In RGST, each training sample is constructed as an input-output pair, where the input consists of the original query q and its gold relevant-passages $\mathcal{P}^+$, and the target output is the teacher-synthesized reasoning query q'. We use the `ms-swift`[2] framework to fine-tune Qwen2.5-7B-Instruct[3] (Yang et al., 2024) on 8 A800 GPUs, and the whole stage takes approximately 2 hours. Training is conducted for 3 epochs with a per-device batch size of 2 and 4 gradient accumulation steps. The learning rate is set to 5×10^{-6}. To improve training efficiency and memory scalability, we adopt DeepSpeed ZeRO-3 (Rajbhandari et al., 2020) and FlashAttention (Dao et al., 2022).

The training data are synthesized from `reasonrank_data_13k` using DeepSeek-R1. For each sample, the teacher model takes the original query together with its gold relevant-passages as input and generates a reasoning query as the supervision target. The same prompt template is used for both data synthesis and RGST training, as shown in Figure 3.

[1] https://huggingface.co/datasets/liuwenhan/reasonrank_data_13k

[2] https://github.com/modelscope/ms-swift

[3] https://huggingface.co/Qwen/Qwen2.5-7B-Instruct

System Prompt

You are a query-understanding LLM, an intelligent assistant capable of fully comprehending a query and completing the given task.

Given a query and its relevant passages, think step by step to reason what should be included in the relevant passages.

Note that you should not try to answer or solve the query, just provide your reasoning process.

Relevant Passages:
{passages_block}

Query:
{query}

Figure 3: RGST prompt of data synthesis and training.

RADR. In RADR, we continue reinforcement learning from the RGST-trained rewriter. During rollout, the rewriter receives only the original query q and autoregressively samples a candidate group $\mathcal{G}(q)$, while the gold relevant-passages $\mathcal{P}^+$ are hidden from the policy and used only for reward computation. We further train the RGST checkpoint with GRPO under the `ms-swift` framework on 8 A800 GPUs, and this stage takes approximately 7 hours. During training, a per-device batch size of 1 is used together with 8 gradient accumulation steps, while 8 sampled generations are drawn for each query to estimate the group-relative advantage. The learning rate is set to 1×10^{-6}, the KL coefficient β is set to 0.001, and the sampling temperature is 0.8. DeepSpeed ZeRO-3 is also adopted in this stage. The two models used to compute the listwise and pointwise rewards are deployed on a separate server with 2 H100 GPUs and communicate with the training process through a `vLLM`[4] server. Specifically, we use Qwen3-30B-A3B as the fixed LLM reranker for the listwise reward and Qwen3-8B as the LLM-based relevance judge for the pointwise reward.

The RADR training data are also constructed from `reasonrank_data_13k`. To align the policy input with the inference setting, the rewriter only observes the original query during generation, while the gold relevant-passages are hidden. The corresponding prompt template for query rewriting is shown in Figure 4. The prompt templates for listwise reranking and pointwise relevance judgment are shown in Figure 5 and Figure 6, respectively.

System Prompt

You are a query-understanding LLM, an intelligent assistant capable of fully comprehending a query and completing the given task.

Given a query, think step by step to reason what should be included in the relevant passages.

Note that you should not try to answer or solve the query, just provide your reasoning process.

Query:
{query}

Figure 4: RADR training and QReason inference rewriting prompt template.

B.3 Experimental Setup and Inference Details

For inference, both the rewriter and the reranker are deployed with the `vLLM` framework, and all experiments are conducted on 4 A800 GPUs. During inference, the rewriter uses the same prompt template as that adopted for rewriting in the RADR stage, as shown in Figure 4. The reranker follows the same prompt template as the LLM reranker used in the listwise reward of RADR, as shown in Figure 5. In all experiments, we concatenate the rewritten query after the original query and use the resulting query for reranking.

For the experiments reported in Table 1, we use a unified evaluation environment to ensure fair comparison. For General LLM Rerankers, we perform listwise reranking with a window size of 20 and a step size of 10. For reranker baselines, we follow the configurations reported in the corresponding papers and reproduce all methods under the same environment. For query rewriting methods, we directly use the reasoning queries generated by TongSearch (7B) and released in its official GitHub repository[5]. For DeepRetrieval, we follow its official GitHub[6] implementation and locally reproduce the query rewriting process, while GPT-4 rewrites are taken from the rewritten queries provided in the BRIGHT test set.

[4] https://github.com/vllm-project/vllm

[5] https://github.com/bigai-nlco/TongSearch-QR

[6] https://github.com/pat-jj/DeepRetrieval

System Prompt

You are RankLLM, an intelligent assistant that can rank passages based on their relevance to the query.

I will provide you with *{num}* passages, each indicated by a numerical identifier [].

Rank the passages based on their relevance to the search query: *{query}*.

{passages_block}

Rank the *{num}* passages above based on their relevance to the search query. All the passages should be included and listed using identifiers, in descending order of relevance. The output format should be [] > [], e.g., [2] > [1]. Only respond with the ranking results, do not say any word or explain.

Search Query: *{query}*.

Figure 5: RADR LLM reranker and QReason inference reranking prompt template.

System Prompt

Judge whether the document is relevant to the query.

Label the provided document as "Relevant" if it contains potentially useful information for addressing the query; otherwise, label it as "Irrelevant."

Output exactly one word: Relevant or Irrelevant. Do not output any explanation or extra text.

Document:
{relevant_passage}

Query:
{rewritten_query}

Figure 6: RADR relevance judge model prompt template.

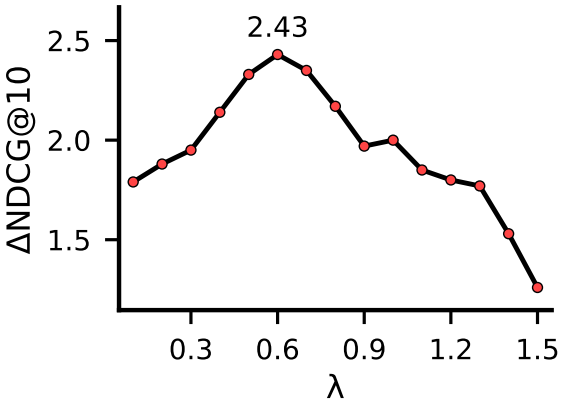


Figure 7: Effect of the reward weight λ.

The reranking configurations used in Tables 2 and 3 are consistent with those in Table 1. In particular, for the efficiency analysis in Table 3, ReasonRank (32B) is evaluated with the same window size and step size as our method.

In all experiments, we set the decoding temperature of both the rewriter in QReason and the reranker to 0 and use greedy decoding. Therefore, the results are nearly identical across multiple runs.

C Additional Experimental Details

C.1 Parameter Sensitivity

As shown in Figure 7, where the y-axis denotes the NDCG@10 gain of reranking with rewritten queries over original queries, performance first improves and then declines as λ increases, with the best result at $\lambda = 0.6$.

When λ is small, the pointwise reward contributes little to the overall optimization, causing the listwise reward to dominate. As a result, the rewriter is more likely to overfit to the preference of the fixed reranker, which may lead to semantic drift in the generated reasoning query q'. In contrast, when λ becomes too large, the stronger pointwise reward weakens the contribution of the listwise reward, making the rewritten queries less aligned with the reranking objective.

C.2 Empirical Efficiency Analysis

The latency comparison on the full set of datasets is reported in Table 5, which also presents the latency of the rewriter in QReason. The average latency of the QReason rewriter is only about $1/65$ of that of ReasonRank (32B) and accounts for approximately 3.9% of the total latency of QReason with Qwen2.5-32B-Instruct. This suggests that the overall runtime of QReason is still dominated by reranking rather than rewriting, making the additional rewriting step highly cost-effective in practice.

D More Experiments

In this section, we provide additional experiments to further evaluate the effectiveness and generalizability of QReason under broader settings, complementing the main results presented in the paper. All results in this section are evaluated using NDCG@10.

D.1 Extended Main Results on BRIGHT

As shown in Table 6, we further evaluate QReason with additional general LLM rerankers to examine whether its benefit transfers across different downstream models. Specifically, we instantiate the downstream reranker with Qwen2.5-72B-Instruct[7] (Yang et al., 2024), Llama3.3-70B-Instruct[8] (Grattafiori et al., 2024), Qwen3-32B[9] (Yang et al., 2025a), Qwen3-30B-A3B[10], and Qwen3.6-35B-A3B[11] (Qwen Team, 2026b), and compare reranking with original queries against reranking with QReason-generated reasoning queries under the same setting. The results demonstrate that the reasoning queries produced by QReason generalize well across different models and consistently yield clear performance gains.

D.2 Performance of QReason with Smaller General LLM Rerankers

We observe that smaller general LLM rerankers show limited ranking effectiveness under the main experimental setting, which may be attributed to their lack of task-specific reranking training. To better examine whether QReason can benefit these smaller rerankers, we conduct additional experiments with a reduced sliding-window configuration. Specifically, we instantiate the downstream reranker with Qwen2.5-7B-Instruct[12] (Yang et al., 2024), Llama-3.1-8B-Instruct[13] (Grattafiori et al., 2024), Qwen3-8B[14] (Yang et al., 2025a), Qwen3.5-9B[15] (Qwen Team, 2026a), Qwen2.5-14B-Instruct[16], and Qwen3-14B[17]. For each reranker, we compare reranking with original queries against reranking with QReason-generated reasoning queries. We set the window size to 10 and the step size to 5, while keeping all other settings the same as those used in Table 1. The results are shown in Table 7.

D.3 Results on Additional Benchmarks

We further evaluate QReason on two additional benchmarks, FreshStack (Thakur et al., 2025) and NeuCLIRBench (Lawrie et al., 2025), to examine its generalization beyond BRIGHT.

FreshStack. FreshStack is a benchmark suite for technical-document retrieval constructed by a reusable IR/RAG benchmark construction framework. It is built from community-asked Stack Overflow questions and technical corpora collected from public GitHub repositories, and covers five fast-growing domains: LangChain, Yolo v7/v8, Laravel 10/11, Angular 16/17/18, and Godot4. It provides nugget-level support judgments for document chunks, making it suitable for evaluating retrieval systems on long, code-intensive technical queries. We use Qwen3-Embedding-8B (Zhang et al., 2025b) as the first-stage retriever and rerank the top-100 retrieved passages.

As shown in Table 8, QReason consistently improves all three LLM rerankers over their original-query settings, with gains of 2.23, 2.53, and 2.26 points on Qwen2.5-32B-Instruct, Qwen3-30B-A3B-Instruct-2507, and Qwen3.5-35B-A3B, respectively. With Qwen3.5-35B-A3B, QReason achieves an average NDCG@10 of 33.63, only 0.05 points below ReasonRank (32B). This result is notable because ReasonRank repeatedly generates reasoning traces during reranking, whereas QReason generates the reasoning query only once and subsequently performs non-reasoning reranking.

NeuCLIRBench. NeuCLIRBench is a multilingual retrieval benchmark that supports monolingual, cross-language, and multilingual retrieval. It contains documents originally written in Chinese, Persian, and Russian, together with English machine translations, and combines topics from the TREC NeuCLIR tracks. The monolingual setting evaluates retrieval in English, Persian, Russian, and Chinese; the cross-language setting uses English

[7] https://huggingface.co/Qwen/Qwen2.5-72B-Instruct
[8] https://huggingface.co/meta-llama/Llama-3.3-70B-Instruct
[9] https://huggingface.co/Qwen/Qwen3-32B
[10] https://huggingface.co/Qwen/Qwen3-30B-A3B
[11] https://huggingface.co/Qwen/Qwen3.6-35B-A3B
[12] https://huggingface.co/Qwen/Qwen2.5-7B-Instruct
[13] https://huggingface.co/meta-llama/Llama-3.1-8B-Instruct
[14] https://huggingface.co/Qwen/Qwen3-8B
[15] https://huggingface.co/Qwen/Qwen3.5-9B
[16] https://huggingface.co/Qwen/Qwen2.5-14B-Instruct
[17] https://huggingface.co/Qwen/Qwen3-14B

queries to retrieve documents in each of the three non-English languages; and the multilingual setting retrieves documents across all three languages with English queries.

Table 9 shows that QReason also transfers consistently to these more traditional multilingual information-retrieval settings. It improves the average performance of all three evaluated rerankers, from 29.73 to 31.13 (+1.40) with Qwen2.5-32B-Instruct, from 30.24 to 31.11 (+0.87) with Qwen3-30B-A3B-Instruct-2507, and from 30.68 to 31.61 (+0.93) with Qwen3.5-35B-A3B. The best configuration, QReason with Qwen3.5-35B-A3B, achieves an average NDCG@10 of 31.61, outperforming ReasonRank (32B) by 1.01 points. Although the gains are smaller than those on BRIGHT and FreshStack, their consistency across all three rerankers indicates that ranking-oriented query reasoning remains beneficial even for less reasoning-intensive monolingual and cross-lingual retrieval tasks.

E Case Details

We provide the detailed information for the case presented in Section 5.7, including the complete original query, the gold relevant-passages, the query rewritten by QReason, and the query rewritten by TongSearch (7B), as shown in Figures 8–14. For readability, URLs in the relevant passages are replaced with *[URL]*.

F Use of AI Assistants

We use ChatGPT to polish the writing of the paper.[18]

G Artifact Licenses and Intended Use

All datasets, benchmarks, pretrained models, and software packages used in this work are publicly available. We use them for research purposes and follow their corresponding licenses and terms of use. The datasets and benchmarks, including BRIGHT, FreshStack and `reasonrank_data_13k`, are used under the terms specified by their original providers. The pretrained models, including Qwen, Llama, and DeepSeek models, are used according to their corresponding model licenses or terms of use. The software packages used in our experiments, including `vLLM` and `ms-swift`, are used under their respective open-source licenses.

We use existing artifacts only for research purposes, including model training, reranking evaluation, and reproducibility analysis, which is consistent with their intended research or benchmarking use where specified. The artifacts created in this work, including the trained QReason rewriter and generated reasoning queries, are intended for research on query rewriting and passage reranking.

[18] https://chatgpt.com/

Table 5: Full end-to-end latency comparison on all BRIGHT subsets. For QReason, the latency includes both the one-time query rewriting cost and the downstream reranking cost. The rewriter latency is also reported separately. Unit: seconds.

Dataset	ReasonRank (32B)	QReason + Qwen3.5-35B-A3B	QReason + Qwen2.5-32B-Instruct	Rewriter
economic	1056.08	163.83	503.97	21.23
earth_science	873.16	181.45	369.29	18.98
robotics	1032.78	156.06	434.76	16.75
biology	754.67	95.89	277.92	18.27
psychology	1121.48	180.53	494.65	18.90
stackoverflow	2154.46	222.97	817.17	15.51
sustainable_living	820.03	130.01	368.01	17.71
leetcode	3385.11	315.04	1193.75	28.60
pony	943.19	116.84	349.46	22.59
aops	1742.63	228.83	663.01	42.78
theoremqa_questions	3437.99	552.65	1313.49	37.44
theoremqa_theorems	1188.31	205.18	456.79	26.33
avg	1542.49	212.44	603.52	23.76

Table 6: Additional results with general LLM rerankers on BRIGHT. Each pair compares reranking with original queries and QReason-rewritten queries under the same reranker.

Reranker	Econ.	Earth.	Rob.	Bio.	Psy.	Stack.	Sus.	Leet.	Pony	AoPS	TheoQ.	TheoT.	Avg
Qwen2.5-72B-Instruct													
with Original Queries	29.70	41.31	24.70	47.22	45.15	28.66	38.89	22.43	21.56	5.34	31.50	43.52	31.67
with **QReason**	33.17	43.56	29.88	50.00	46.81	29.00	43.75	23.41	30.41	7.31	36.35	45.32	34.91 (↑3.24)
Llama3.3-70B-Instruct													
with Original Queries	29.74	43.26	23.70	49.95	45.90	27.55	39.86	22.53	22.75	7.27	31.45	36.88	31.74
with **QReason**	30.13	45.22	27.34	50.02	42.74	27.91	42.01	25.17	29.88	8.40	35.77	41.85	33.87 (↑2.13)
Qwen3-32B													
with Original Queries	31.83	43.34	28.12	49.02	46.98	27.33	42.43	28.31	14.80	8.05	38.12	42.39	33.39
with **QReason**	32.99	47.39	29.59	50.33	45.25	31.51	44.69	26.95	23.72	8.28	38.78	44.94	35.37 (↑1.98)
Qwen3-30B-A3B													
with Original Queries	31.82	44.91	23.31	52.77	44.62	32.01	38.00	30.10	20.89	8.77	36.81	38.61	33.55
with **QReason**	31.74	48.23	25.27	53.59	45.56	33.44	38.61	29.46	24.64	9.18	38.29	44.36	35.20 (↑1.65)
Qwen3.6-35B-A3B													
with Original Queries	33.08	45.94	28.55	51.89	48.11	28.18	39.97	30.65	29.85	9.16	40.44	40.55	35.53
with **QReason**	34.59	47.50	29.27	54.20	46.14	30.45	42.94	29.99	31.72	10.18	43.89	44.83	37.14 (↑1.61)

Table 7: Performance of smaller general LLM rerankers on BRIGHT under a window size of 10 and a step size of 5.

Reranker	Econ.	Earth.	Rob.	Bio.	Psy.	Stack.	Sus.	Leet.	Pony	AoPS	TheoQ.	TheoT.	Avg
Qwen2.5-7B-Instruct													
with Original Queries	28.32	40.78	20.64	42.12	39.78	26.89	30.12	30.27	19.40	4.23	28.97	37.09	29.05
with **QReason**	27.69	42.63	23.21	44.62	39.43	26.97	31.71	31.05	24.38	6.14	32.47	38.89	30.77 (↑1.72)
Llama-3.1-8B-Instruct													
with Original Queries	23.97	36.04	20.78	43.89	36.37	21.22	27.75	29.82	18.35	6.59	23.81	27.02	26.30
with **QReason**	24.13	36.43	22.38	48.82	38.43	27.57	30.55	27.28	21.03	7.49	26.98	28.03	28.26 (↑1.96)
Qwen3-8B													
with Original Queries	29.53	43.00	25.03	49.47	38.22	26.37	33.71	30.48	25.08	7.68	34.49	40.20	31.94
with **QReason**	29.07	46.23	25.42	52.44	40.51	27.28	35.58	30.61	35.18	9.41	36.74	42.55	34.25 (↑2.31)
Qwen3.5-9B													
with Original Queries	33.18	46.34	26.79	48.72	41.71	31.02	37.67	31.91	21.90	8.31	41.50	40.69	34.15
with **QReason**	33.41	47.34	29.63	52.58	42.45	32.05	39.14	34.21	24.29	9.43	42.56	45.13	36.02 (↑1.87)
Qwen2.5-14B-Instruct													
with Original Queries	29.00	41.73	23.20	48.20	43.59	28.10	37.71	23.18	22.45	7.78	27.48	37.46	30.82
with **QReason**	29.41	44.49	24.75	49.14	44.74	27.85	37.17	24.51	26.40	10.25	33.46	40.83	32.75 (↑1.93)
Qwen3-14B													
with Original Queries	31.97	44.15	27.21	51.79	43.25	29.19	39.07	33.07	26.17	7.93	37.69	40.03	34.29
with **QReason**	33.24	46.36	28.06	54.34	44.30	30.77	37.75	35.93	27.07	9.67	39.37	42.21	35.76 (↑1.47)

Table 8: Additional results on FreshStack. The best score is shown in **bold** and the second best is underlined.

	LangChain	Yolo	Laravel	Angular	Godot	Avg
Qwen3-Embedding-8B	25.34	32.16	34.24	28.53	26.81	29.42
ReasonRank (32B)	31.52	**37.11**	36.73	31.78	31.24	**33.68**
Qwen2.5-32B-Instruct						
with Original Queries	29.97	28.23	32.26	29.72	28.46	29.73
with **QReason**	**31.76**	30.12	33.58	32.13	**32.22**	31.96 (↑2.23)
Qwen3-30B-A3B-Instruct-2507						
with Original Queries	25.34	32.16	34.30	28.53	26.83	29.43
with **QReason**	27.85	34.41	37.08	29.57	30.88	31.96 (↑2.53)
Qwen3.5-35B-A3B						
with Original Queries	29.32	31.82	36.96	30.03	28.70	31.37
with **QReason**	30.55	35.18	**39.01**	**32.99**	30.41	33.63 (↑2.26)

Table 9: Additional results on NeuCLIRBench. The best score is shown in **bold** and the second best is underlined.

	Monolingual				**Cross-Lingual**			**MLIR**	Avg
	Eng	Fas	Rus	Zho	Fas	Rus	Zho		
BGE-M3 Sparse	31.90	36.74	28.16	27.29	4.11	5.09	8.57	5.61	18.43
ReasonRank (32B)	44.53	52.67	48.32	45.19	9.11	12.14	20.05	12.82	30.60
Qwen2.5-32B-Instruct									
with Original Queries	44.08	53.02	47.39	42.13	8.28	11.10	19.01	12.79	29.73
with **QReason**	44.19	55.21	48.90	45.84	8.66	**12.83**	**22.15**	11.29	31.13 (↑1.40)
Qwen3-30B-A3B-Instruct-2507									
with Original Queries	44.83	53.90	47.57	43.78	8.36	11.39	19.46	12.60	30.24
with **QReason**	**45.49**	54.43	48.62	44.88	**9.32**	12.70	20.18	13.22	31.11 (↑0.87)
Qwen3.5-35B-A3B									
with Original Queries	43.11	55.46	48.81	45.54	8.68	11.83	19.50	12.53	30.68
with **QReason**	44.73	**56.13**	**50.28**	**46.22**	9.18	12.16	20.95	**13.26**	**31.61** (↑0.93)

Original Query

How could you partition Household Food Expenditures into Producing Industries?
From the 2019 Consumer Expenditure Survey, the average household annual food expenditure was $8,169 for that year. There are subcategories such as "Food at Home" and "Food away from Home," but for simplicity, I'll keep the scope to annual household food expenditures.
I want to partition this $8,169 into the BEA industries (aggregation level is "summary") for the ultimate goal of assessing pollution impact.

Figure 8: Complete original queries for the case in Section 5.7.

Gold Relevant Passages [1]
[1] Data Files
Supply Tables
Domestic supply of commodities by industry

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Use Tables
Use of commodities by industry

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Import Matrices/Before Redefinitions
Use of imported commodities by industry

- 2017–2022: [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Figure 9: Gold relevant-passages [1] for the case in Section 5.7.

Gold Relevant Passages [2]
[2]

- 2017–2022: [15 Industries] (*[URL]*) XLSX, [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Use Tables/After Redefinitions/Purchaser Value
Use of commodities by industry valued at purchasers' prices

- 2017: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable, [402 Industries] (*[URL]*) XLSX

The after redefinition tables are used as inputs into the derivation of the total requirements and domestic requirements matrices. Additional details regarding the preparation of the after redefinition tables can be found in the *Total Requirements Derivation* (*[URL]*) and *Domestic Requirements Derivation* (*[URL]*) documentation.
Make Tables/Before Redefinitions

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Use Tables/Before Redefinitions/Producer Value

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Use Tables/Before Redefinitions/Purchaser Value

- 2017: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Make Tables/Before Redefinitions

- 1963–1996: [65 Industries] (*[URL]*) XLSX
- 1947–1962: [46 Industries] (*[URL]*) XLSX

Use Tables/Before Redefinitions/Producer Value

- 1963–1996: [65 Industries] (*[URL]*) XLSX
- 1947–1962: [46 Industries] (*[URL]*) XLSX

Figure 10: Gold relevant-passages [2] for the case in Section 5.7.

Gold Relevant Passages [3]
[**3**] Supplemental Estimate Tables

- Requirements Tables
- Make-Use Tables
- Historical Make-Use Tables
- Total Requirements
- Domestic Requirements
- After Redefinition Tables

Commodity by Commodity/After Redefinitions/Producer Value
Total inputs by commodity required (directly and indirectly) in order to deliver one dollar of commodity output to final users

- 2017–2022: [17 Commodities] (*[URL]*) iTable, [73 Commodities] (*[URL]*) iTable
- 2017: [402 Commodities] (*[URL]*) XLSX

Industry by Commodity/After Redefinitions/Producer Value
Total inputs by industry required (directly and indirectly) in order to deliver one dollar of commodity output to final users

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Industry by Industry/After Redefinitions/Producer Value
Total inputs by industry required (directly and indirectly) in order to deliver one dollar of industry output to final users

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Additional information regarding the *Total Requirements Derivation* (*[URL]*).
Domestic requirements tables were first released by BEA on Feb. 17, 2017. Additional details about these statistics are available in the *Survey of Current Business*: *Introducing Domestic Requirements Tables for 1997–2015* (*[URL]*).
Commodity by Commodity/After Redefinitions/Producer Value
Domestic inputs by commodity required (directly and indirectly) in order to deliver one dollar of commodity output to final users

- 2017–2022: [17 Commodities] (*[URL]*) XLSX, [73 Commodities] (*[URL]*) XLSX
- 2017: [402 Commodities] (*[URL]*) XLSX

Industry by Commodity/After Redefinitions/Producer Value
Domestic inputs by industry required (directly and indirectly) in order to deliver one dollar of commodity output to final users

Figure 11: Gold relevant-passages [3] for the case in Section 5.7.

Gold Relevant Passages [4]
[4]

- 2017–2022: [15 Industries] (*[URL]*) XLSX, [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Industry by Industry/After Redefinitions/Producer Value
Domestic inputs by industry required (directly and indirectly) in order to deliver one dollar of industry output to final users

- 2017–2022: [17 Industries] (*[URL]*) XLSX, [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Additional information regarding the *Domestic Requirements Derivation* (*[URL]*).
Make Tables/After Redefinitions
Production of commodities by industry after redefinition of secondary production

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Market Share Tables/After Redefinitions
Share of commodity production by after redefinition industry

- 2017–2022: [15 Industries] (*[URL]*) XLSX, [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Use Tables/After Redefinitions/Producer Value
Use of commodities by industry after reallocation of inputs associated with redefined secondary production

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Import Matrices/After Redefinitions
Use of imported commodities by industry

- 2017–2022: [71 Industries] (*[URL]*) XLSX
- 2017: [402 Industries] (*[URL]*) XLSX

Direct Requirements/After Redefinitions/Producer Value
Total inputs by commodity directly required in order to produce one dollar of industry output

- 2017–2022: [15 Industries] (*[URL]*) iTable, [71 Industries] (*[URL]*) iTable
- 2017: [402 Industries] (*[URL]*) XLSX

Domestic Direct Requirements/After Redefinitions/Producer Value
Domestic inputs by commodity directly required in order to produce one dollar of industry output

Figure 12: Gold relevant-passages [4] for the case in Section 5.7.

QReason Rewrite

The query asks how household food expenditures reported in the Consumer Expenditure Survey can be translated into BEA industry-level production attributions. This requires identifying an accounting framework that connects consumer spending, final demand, directly purchased food sectors, and upstream industry contributions through input-output relationships.

Relevant passages should likely include:

1. **Economic accounting frameworks and input-output modeling** (e.g., Input-Output analysis)
2. **Industry classification systems** (e.g., NAICS/BEA industry codes)
3. **Supply and use tables**, including related **industry input-output linkages**
4. **Intermediate vs. final demand** and the **attribution of consumption to production processes**
5. **Expenditure flows and their decomposition** across a **multi-sector economy**
6. **Accounting for supply chains and value creation**
7. **National accounting systems** (e.g., System of National Accounts)
8. **Expenditure surveys methodology**
9. **Value added, production, and accounting for value transformation**
10. **Distribution of economic activity**

Figure 13: Queries rewritten by QReason for the case in Section 5.7.

TongSearch-7B Rewrite

To partition household food expenditures into producing industries using the Bureau of Economic Analysis (BEA) industries, consider the following approach:

Essential Problem:

The core issue is to accurately allocate $8,169 in household food expenditures across various BEA industry summaries, specifically focusing on industries like agriculture (farms), forestry, fishing, food manufacturing, retail, and food services. The objective is to assess pollution impacts accurately by tracing these expenditures through supply chains.

Step-by-Step Reasoning:

1. **Understand the Data and Context:**
   - **Household Expenditures:** Total $8,169 annually.
   - **BEA Industry Summary Codes:**
     - 111CA: Farms
     - 113FF: Forestry, Fishing & Related activities
     - 311FT: Food and Beverage Manufacturing
     - 445: Food and Beverage Stores
     - 722: Food Services and Drinking Places
2. **Identify Relevant Subcategories:**
   - "Food at Home" and "Food Away from Home" can be mapped to different industries.
   - For example, "Food at Home" typically involves grocery stores (445), while "Food Away from Home" includes restaurants (722).
3. **Input-Output Model Considerations:**
   - The Leontief Input/Output model suggests tracing upstream inputs, which aligns with allocating expenditures to industries based on production and supply chains.
   - However, households do not directly purchase intermediate goods; they buy final products.
4. **Attribution Method:**
   - **Direct Allocation:** Start by allocating expenditures directly to the industries where final goods are sold.
     - Assume a significant portion goes to 311FT (Food and Beverage Manufacturing).
     - A smaller portion might go to 445 (Food and Beverage Stores).
     - Some may also go to 722 (Food Services).
   - **Indirect Contributions:** Use industry-specific input-output tables to trace back to primary producers (e.g., 111CA for agricultural inputs).
5. **Account for Value Added:**
   - Consider the value added by each industry in the production process.
   - For instance, if a loaf of bread is sold to a household, the farm (111CA) contributes to the initial production, while the bakery (311FT) adds

Figure 14: Queries rewritten by TongSearch (7B) for the case in Section 5.7.